$r_{k+1} = \delta(r_k, \omega_{k+1})$ (sequence of states visited)

$k \in \{0, 1, \ldots, m-1\}$

Since $m \geq p$, consider the sequence $\underbrace{r_0, r_1, \ldots, r_p}_{p+1 \text{ states}}$

Recall the DFA M has only p states. (since $|Q| = p$) } Hence in the sequence we must revisit a state.

So, there must exist indices s & $t \in \{0, 1, \ldots, p\}$, $s < t$, where $r_s = r_t$.

The state that we revisit in the cycle.

This allows us to fix

$x = \omega_1 \omega_2 \ldots \omega_s$, $y = \omega_{s+1} \ldots \omega_t$, $z = \omega_{t+1} \ldots \omega_p$

y: string processed in the cycle

i) Since $t > s$, y cannot be λ.

ii) xy has length t & $t \leq p$ since $t \in \{0, 1, \ldots p\}$. So, $|xy| \leq p$

iii) Finally, $xy^i z \in A$ means M accepts $xy^i z$ for all $i \in \{0, 1, \ldots\}$

Recall $\underbrace{r_0, r_1, \ldots r_s}_{x}, \underbrace{r_{s+1}, \ldots, r_t}_{y}, \underbrace{r_{t+1}, \ldots, r_m}_{z}$ is the accepting path.

$r_{s+1}, \ldots, r_t$ can repeat i times since $r_s = r_t$

Regular Expressions (RE)

Another model that recognizes regular languages.

Def

R is a regular expression if R is

1. $a \in \Sigma$
2. λ
3. ϕ

} Base Cases

4. $R_1 \cup R_2$ where R_1 & R_2 are regular expressions
5. $R_1 \cdot R_2$
6. R_1^* where R_1 is a regular expression.

} Recursive def.

e.g. $(a+b)^* \cdot (ab + ba) \cdot (a+b)^*$ has language

$L = \{ \omega \in \Sigma^* \mid \omega \text{ contains } ab \text{ OR } ba \}$ for $\Sigma = \{a, b\}$

There exist algorithm to convert RE to NFA & back. You should understand how & why they work.

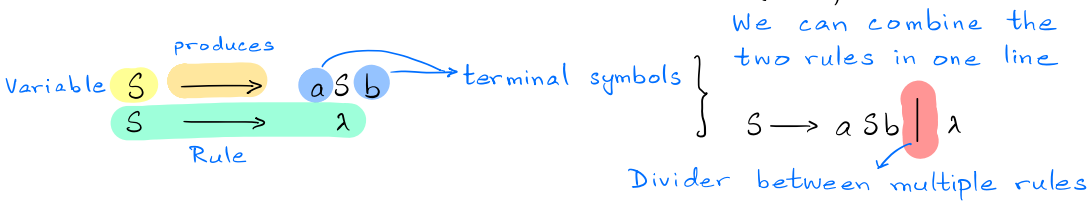
Contex Free Languages (CFL)

Consider the non-regular language

$L = \{ a^n b^n \text{ for } n \geq 0 \}$, $\Sigma = \{a, b\}$

$S \rightarrow aSb$
$S \rightarrow \lambda$

The strings in L can be generated by the following two rules of a Context Free Grammar (CFG).



Def A CFG G is a 4-tuple (V, Σ, R, S) where

1. V is a finite set of variables (denoted by capital letters usually)
2. Σ is the alphabet with disjoint symbols from V (terminal symbols)
3. R is a finite set of rules with a single variable on LHS & a string of variables & terminals on RHS
4. $S \in V$ is a start variable (unique)

We generate or produce strings by starting with the starting variable S and substituting it with a string on the RHS of a rule with S on the LHS. This is a single step of a derivation.

We continue by replacing variables in the new string with any rule till we are left with a string with no variables.
The resulting string is produced by the CFG.
e.g., in the above case:

derives

1. $S \Rightarrow \lambda$ So, $\lambda \in$ Language of G

2. $S \Rightarrow aSb \Rightarrow a\lambda b = ab$

3. $S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\lambda bb = aabb$

$S \overset{*}{\Rightarrow} aabb$
↳ one or more steps of substitution.

Note how a's & b's are matched in derivation process.
Something we couldn't do in a DFA etc.

Language of G is all the strings derived from S.

$L(G) = \{ w \in \Sigma^* \text{ where } S \overset{*}{\Rightarrow} w \}$

This is equivalent to an NFA with access to a stack (PDA) where symbols can be pushed or popped with each transition.

e.g.
PALINDROMES = $L = \{ w \in \Sigma^* \text{ where } w = w^R \}$, $\Sigma = \{a, b\}$ (→ Reverse)

so, a, b, aa, aabaa, bab $\in L$
ba, ab, bba $\notin L$

$S \rightarrow \lambda \mid a \mid b \mid aSa \mid bSb$

- λ: even length string base case
- $a \mid b$: odd length base case
- $aSa \mid bSb$: recursive rules that match symbols

e.g.,
$BAL = \{ w \in \Sigma^* \text{ where } w \text{ has matching brackets} \}$, $\Sigma = \{ (,) \}$ (alphabet symbols)

so, ()(), (()), ((())) $\in$ BAL
(,)(, ()) $\notin$ BAL

$S \rightarrow \lambda \mid (S) \mid SS$

- (S): matched brackets
- SS: allow different blocks of matched brackets

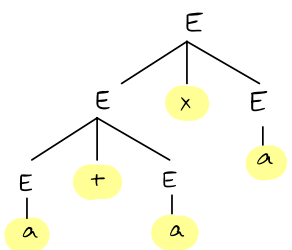
Ambiguity

A CFG is ambiguous if there exist two different parse trees or two left-most derivations for the same string.

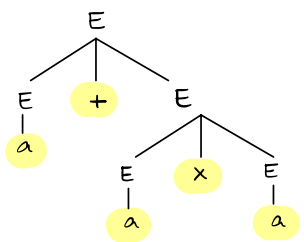
e.g.,
$E \rightarrow a \mid (E) \mid ExE \mid E+E$, $\Sigma = \{ a, +, x, (,) \}$

Consider $w = a+a \times a$

Parse Tree 1



Parse Tree 2



Can make G unambiguous by enforcing precedence, e.g.,

$E \rightarrow E+T \mid T$
$T \rightarrow T \times F \mid F$
$F \rightarrow (E) \mid a$

Inherently Ambiguous if language can only be generated via an ambiguous grammar, e.g.,

$L = \{ a^i b^j c^k \text{ where } i=j \text{ OR } j=k \}$ Try building the grammar.

CFL are closed under union but not intersection. Why?

For closure under union, given two CFGs $G_1 = (V_1, \Sigma_1, R_1, S_1)$ & $G_2 = (V_2, \Sigma_2, R_2, S_2)$ we define

$$G_3 = (V_1 \cup V_2 \cup \{S_3\}, \Sigma_1 \cup \Sigma_2, R_1 \cup R_2 \cup \{S_3 \to S_1 \mid S_2\}, S_3)$$

(S_3: new starting variable)

($S_3 \to S_1 \mid S_2$: new rule that allows generating strings of both G_1 & G_2)

Note $L(G_3) = L(G_1) \cup L(G_2)$

Hence, CFLs are closed under union.

Consider $L = \{a^n b^n c^n \text{ where } n \geq 0\}$. Convince yourself that L is not CFL, by trying to build a CFG for it.

Let $L_1 = \{a^* b^n c^n, n \geq 0\}$ & $L_2 = \{a^n b^n c^*, n \geq 0\}$

Note a^* is generated by $S_1 \to \lambda \mid aS_1$

$b^n c^n$ is generated by $S_2 \to \lambda \mid bS_2c$

} Add rule $S_3 \to S_1 S_2$ to get $L_1 = \{a^* b^n c^n\}$

Similarly, L_2 is also a CFL, but $L = L_1 \cap L_2$. Since L is not a CFL, they are not closed under intersection.

Note however, for a CFL(L_1) ∩ Regular(L_2) = CFL(L_3)

Chomsky Normal Form

$A_{CFG} = \{\langle G, w \rangle \text{ where } G \text{ is a CFG \& } w \text{ is generated by } G\}$

(brackets indicate a fixed encoding of G & w.)

How can we decide whether a given G generates a particular string w?

A CFG is in CNF if every rule is in one of the following form:

i) $A \to BC$ (two variables that are not starting variable)
ii) $A \to a \in \Sigma$ (single terminal symbol)

Thm

Any CFL can be generated by a CFG in CNF.

Perform following steps to convert CFG into CNF.

1. Add $S_0 \to S$ (S_0 is the new starting variable)

   (S is the old starting variable)

2. Remove all λ rules, i.e., $A \to \lambda$
   Replace with corresponding rules on RHS with A removed

3. Remove unit rules, i.e., $A \to B \to u$ is replaced with $A \to u$ (u: string of variables & terminals)

4. Take care of rules with RHS length > 2.

e.g.,

$S \longrightarrow ASB$
$A \longrightarrow aAS \mid a \mid \lambda$
$B \longrightarrow SbS \mid A \mid bb$

1. Add S_0 →

$S_0 \longrightarrow S$
$S \longrightarrow ASB$
$A \longrightarrow aAS \mid a \mid \lambda$
$B \longrightarrow SbS \mid A \mid bb$

2. Remove λ Rules

Remove $A \longrightarrow \lambda$

$S_0 \longrightarrow S$
$S \longrightarrow ASB \mid SB$
$A \longrightarrow aAS \mid a \mid aS$
$B \longrightarrow SbS \mid A \mid bb \mid \lambda$

We ended up adding $B \longrightarrow \lambda$. Remove it.

$S_0 \longrightarrow S$
$S \longrightarrow ASB \mid SB \mid S \mid AS$
$A \longrightarrow aAS \mid a \mid aS$
$B \longrightarrow SbS \mid A \mid bb$

3. Remove Unit Rules

Remove $B \longrightarrow A$

$S_0 \longrightarrow S$
$S \longrightarrow ASB \mid SB \mid S \mid AS$
$A \longrightarrow aAS \mid a \mid aS$
$B \longrightarrow SbS \mid bb \mid aAS \mid a \mid aS$

Remove $S \longrightarrow S$

$S_0 \longrightarrow S$
$S \longrightarrow ASB \mid SB \mid AS$
$A \longrightarrow aAS \mid a \mid aS$
$B \longrightarrow SbS \mid bb \mid aAS \mid a \mid aS$

Remove $S_0 \longrightarrow S$

$S_0 \longrightarrow ASB \mid SB \mid AS$
$S \longrightarrow ASB \mid SB \mid AS$
$A \longrightarrow aAS \mid a \mid aS$
$B \longrightarrow SbS \mid bb \mid aAS \mid a \mid aS$

4. Make RHS variable rules have form $A \longrightarrow BC$

e.g., replace

$S_0 \longrightarrow ASB$

with

$S_0 \longrightarrow AU_1$
$U_1 \longrightarrow SB$

Finally, we obtain

$S_0 \longrightarrow AU_1 \mid SB \mid AS$
$S \longrightarrow AU_1 \mid SB \mid AS$
$A \longrightarrow V_1U_2 \mid a \mid V_1S$
$B \longrightarrow SU_3 \mid V_2V_2 \mid V_1U_2 \mid a \mid V_1S$
$U_1 \longrightarrow SB$
$U_2 \longrightarrow AS$
$U_3 \longrightarrow V_2S$
$V_1 \longrightarrow a$
$V_2 \longrightarrow b$

Note, now we can decide A_{CFG} for a string w of length n by converting G into CNF & considering all derivations of length $2n-1$. Why?